

Embedded Monocular Obstacle Avoidance in MAVs Using Extreme Knowledge Distillation

D.J. Rugge, T.M. van Dam, W. van Mildert, M. Frans, J. Slagmolen, T.B.M. van Santen
Technical University of Delft, Mekelweg 5, Delft

ABSTRACT

Autonomous navigation in resource-constrained micro air vehicles (MAVs) demands lightweight perception and robust obstacle avoidance. This project report presents the design and deployment of a monocular vision-based obstacle avoidance system for the Parrot Bebop drone, enabling autonomous flight in the TU Delft Cyber Zoo, as part of the TU Delft course *AE4317: Autonomous Flight of Micro Air Vehicles*. To address the drone’s limited computational power, we propose *hyper distillation*—a method that condenses monocular depth maps into estimated danger grids—paired with a reactive navigation algorithm. The system performs embedded processing using a three-layer convolutional network, achieving inference speeds as low as 20 ms. Despite hardware and testing constraints (only 30 minutes of weekly real-world flight time), the drone flew autonomously for 350 meters in competition, securing second place in the TU Delft Cyber Zoo competition. While some collisions occurred, the drone successfully navigated through the course, demonstrating the effectiveness of our approach.

1 INTRODUCTION

Autonomous obstacle avoidance in Micro Air Vehicles (MAVs) is critical for safe navigation where low latency ground control is infeasible. This project addresses the challenge of enabling embedded monocular obstacle avoidance on the Parrot Bebop. In the project, the drone must navigate autonomously within the boundaries of a safe testing environment—referred to as the Cyber Zoo—while avoiding obstacles using only onboard computation and its built-in sensors. Key challenges include real-time image processing on the drone’s constrained hardware, balancing computational efficiency with obstacle detection accuracy, and limited physical testing opportunities in a tight seven-week deadline. To overcome these, we propose a novel **hyper distillation** method that condenses monocular depth estimation into a lightweight convolutional neural network (CNN). This approach guides navigation by providing simple directional danger scores. Our work consists of three main components:

- **A brief literature review** of lightweight MAV-embedded obstacle avoidance methods, with particular attention to gaps in deployability for resource-constrained drones.
- **Hyper Distillation:** An intense knowledge distillation pipeline that enables depth estimation for the Bebop’s hardware, achieving inference times under 20 ms, enabling high flight speeds.
- **A navigation algorithm** that adjusts flight commands based on real-time danger scores from the rapid depth estimation.

2 BACKGROUND

Educational Context This project was conducted under TU Delft’s *AE4317: Autonomous Flight of Micro Air Vehicles* course, hosted by the Micro Air Vehicle Laboratory (MAVLab). Students are tasked with developing a Paparazzi-based autonomy module for the Parrot Bebop drone to navigate an obstacle course.

Software and Hardware Details

- **Paparazzi UAV Software:** The open source project Paparazzi [1] autopilot (v6.4) provides the basis for drone control, including a Gazebo simulation of the Cyber Zoo (also referred to as NPS - New Paparazzi Simulator).
- **Cyber Zoo Simulation:** A Gazebo model replicates the physical Cyber Zoo environment, allowing for safe testing of the drone’s navigation and obstacle avoidance capabilities.
- **OptiTrack:** A local indoor camera-based positioning system is used to track the drone’s position in the Cyber Zoo both in simulation and real-world testing.
- **Drone Specifications:** The Parrot Bebop drone is equipped with a fisheye front-facing camera and Parrot P7 dual-core CPU Cortex 9 processor, running on Linux (kernel version 3.4.11+).

Project Requirements The solution must adhere to the following core requirements:

- The model must not interfere with the drone’s onboard stabilization modules, as significant inference time can introduce delays in the stabilization loop, leading to loss of control.

- The drone must adapt to a dynamic environment, as MAVLab staff will move objects around during the final competition.
- The drone must be able to fly through the obstacle zone multiple times without crashing, with a minimum distance of 150 m flown.
- The model must process images quickly enough to allow for (relatively) high-speed flight, with a maximum inference time of 100 ms and desired inference time under 25 ms (including preprocessing).

Software Baseline The MAVLab provides a basic framework for autonomous flight, including sample code and exercises. The initial code for avoiding orange objects in guided mode served as the starting point for this project.

3 RELATED WORK

This section presents an exploratory review of literature related to the problem of autonomous obstacle avoidance in the context of the MAV domain. The approaches are briefly discussed and analyzed to identify the most suitable approach for the problem.

3.1 Monocular Depth Estimation

Monocular depth estimation is a popular problem in the field of computer vision with many freely available foundation models like MiDaS [2], NewCRFS [3], DepthAnything [4,5], approaches by the Meta Research group, they call Consistent Depth [6,7], which is a temporally consistent depth estimation model, as well as models designed for lightweight embedded CPU-based inference like [8–16] (though there are not as many trained models available for download in this latter category). These models generally use deep convolutional networks and are trained using some self-supervised method, where either stereo images, videos that allow 3D reconstruction and/or depth measurements from a depth camera are used to generate training data. The estimated depth can then be used to assess safe or dangerous regions in the image. To make these models more lightweight, quantization, fusion, pruning, downscaling, hardware optimizations, and other techniques are used. However, even the lightweight models in the given examples are still quite large, or not optimized for the hardware available on the Bebop, or not available for download to test, which makes assessment of the viability of the model architecture difficult. Deployment options of such models are discussed in further detail in Subsection 3.4.

3.2 Danger from bounding boxes

An object detection model processes camera images to identify and classify objects. Such a model would be beneficial in distinguishing different types of objects and their shapes, which can be used to steer the drone away from these objects. An initial object detection model based on

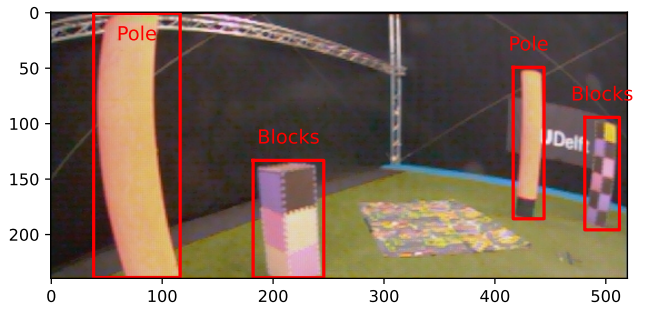


Figure 1: Fine-tuned YOLOv11s predictions

YOLOv11n¹ was fine-tuned with a small dataset of hand-labeled images and showed promising results. In Figure 1, an example image with bounding box predictions is shown. It was attempted to implement such a model on the drone and convert the bounding box predictions into a steering direction on the drone.

However, cross-compiling this model to run on the drone presented large technical difficulties, as is explained in Subsection 3.4. Rather, it was decided to use a much simpler model to predict safe and dangerous zones of the image. For most objects, the size is known a priori. By comparing the width and height of the bounding boxes to the known object dimensions, a measure of distance can be obtained. These danger scores were then used as training data. A simple Convolutional Neural Network can learn these ‘danger scores’ for different regions of the image. In Figure 2, an example with five columns is shown; here, the column is assigned the maximum value of the danger scores of all the objects inside the respective column. This method showed good results, but it was not used in the competition because the model discussed in Section 4 outperformed it in terms of drone speed and inference time.

3.3 End to End Learning

End-to-end learning represents an approach where neural networks learn directly from raw inputs to desired outputs, eliminating the need for manually engineered intermediate steps. In the context of this project, this would mean training an agent that takes the camera images as input and directly outputs control commands. This approach is discussed in more detail in the literature review in [17], but the examples in that review that use the Bebop [18, 19] do not use embedded processing.

While the concept is theoretically appealing, implementing a robust reinforcement learning pipeline for drone navigation presents significant challenges. Given the limited available testing time in the Cyber Zoo and safety considerations, a majority of the training would have to be performed in the NPS simulator. However, while the drone may perform well

¹<https://docs.ultralytics.com/models/yolo11/>

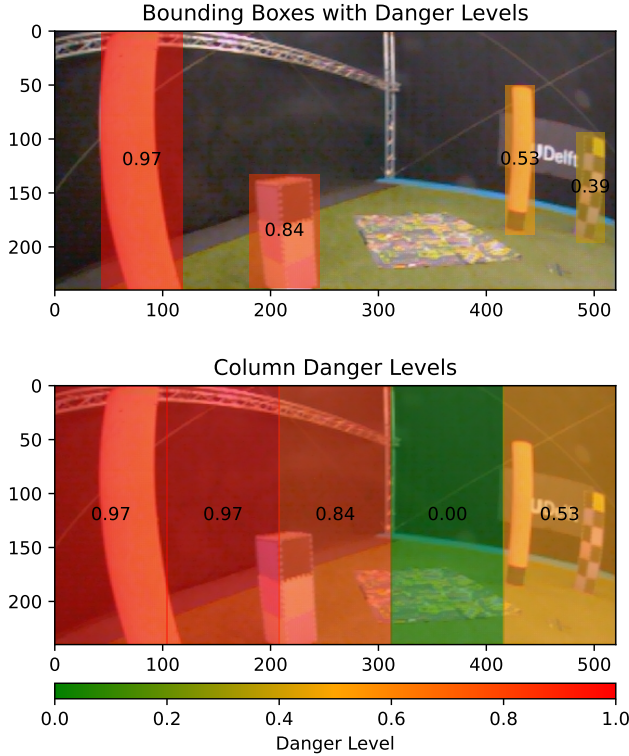


Figure 2: Danger prediction from bounding boxes

in simulation, it may struggle in actual Cyber Zoo conditions due to differences in physics, lighting, textures, and other factors. This problem is also referred to as the sim-real gap.

Advanced domain adaptation techniques like GANs (Generative Adversarial Networks) [20], gaussian splatting [21], or even the more traditional COLMAP [22] could potentially bridge the visual appearance gap by generating realistic training environments based on real data. However, these approaches add considerable complexity to the development pipeline, or simply did not work during exploratory testing, most likely due to the low quality of the images taken by the drone (due to low resolution, frequent motion blur and noise, fisheye lens distortion, and low framerate). Further drawbacks of this approach are that end-to-end learning models are difficult to interpret, making it hard to understand how decisions are made and troubleshoot errors. This complicates the understanding of decision-making processes and presents challenges in troubleshooting errors.

3.4 Machine Learning in C

There are several tools available to compile or otherwise deploy neural networks on edge devices like the Bebop drone, like TFLite, TVM, ONNX2C and NeuralCasting [23–26], among others. The former two, and most tools designed for edge device deployment of neural networks, require compiled libraries, which require newer build tools than are available

on the Bebop. ONNX2C and NeuralCasting are built to compile a neural network to pure C code using only standard libraries, which is a viable approach. Although the NeuralCasting paper reports better optimizations, it supports fewer operators than ONNX2C, which already provides limited operator support. NeuralCasting might be a better choice when it has matured more, but for now, ONNX2C is the best choice for the Bebop drone. However, this poses limitations on the use of existing trained models, as those few that are available for download are generally not supported by ONNX2C. These limitations motivate our focus on knowledge distillation. By distilling dense depth maps from teacher models (e.g., MiDaS) into a lightweight CNN built with the Bebop in mind, we enabled our drone to perform obstacle detection at a reasonable speed. This approach is discussed in more detail in the following Section.

4 HYPER DISTILLATION OF MONOCULAR DEPTH

4.1 Teacher Model Selection

The teacher models initially considered for the distillation process were the MiDaS, UDepth, NewCRFS and Consistent Depth solutions discussed earlier, and in a later stage of the project, also Depth Anything (V2). Of these, Consistent Depth was immediately ruled out. Even though it may produce superior depth maps because of its temporally aware inference, processing a dataset of an hour of video data was estimated to take multiple days on our available GTX 970 GPU. Depth Anything V2 was, unfortunately, only discovered in the final week of the project, so this teacher model was not explored. Of MiDaS, Udepth, and NewCRFS, MiDaS showed the most promising results in the close range indoor environment of the Cyber Zoo. Therefore, it was chosen as the teacher model for the distillation process, though it must be noted that this model struggles with motion blur.

Thus, MiDaS was used to generate the depth maps for the distillation process. The generated depth maps were manually reviewed, and bad sequences resulting from high motion blur or other issues were removed from the dataset. The remaining sequences were used to train the student model.

4.2 Data Preparation and Hyper Distillation

To use the relatively dense depth maps generated by the teacher model effectively, we applied an intense distillation process, which we like to refer to as hyper distillation. The drone is set to its lowest resolution of 520x240 pixels. Then, the input images are cropped to the center 240x240 pixels to focus on the most critical region of the image, that is, where collisions are most likely to occur (coincidentally also avoiding the fish-eye lens distortion). Then, a simplified 'danger value' is generated for three columns in the cropped image:

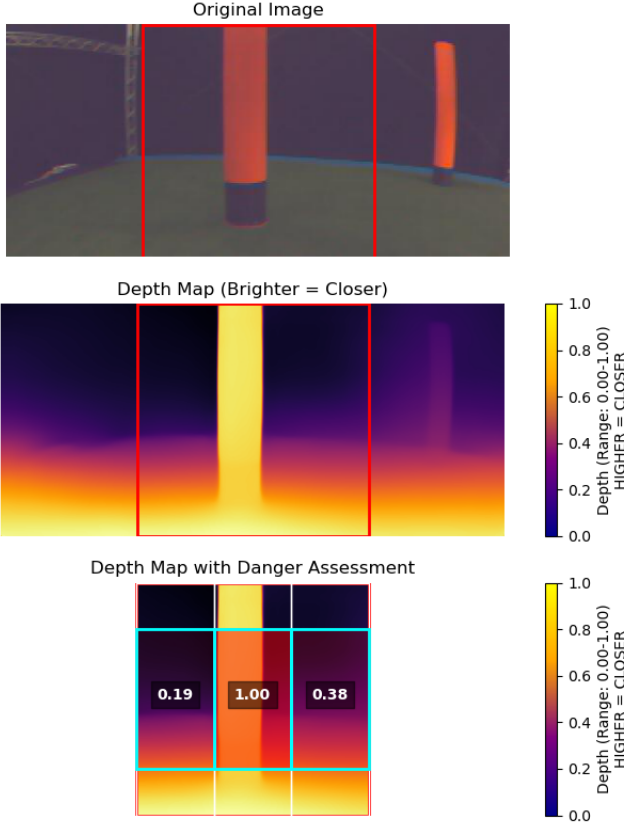


Figure 3: Example of the input image, the associated depth map, the cropping regions, and the generated danger values.

- First, only the vertical center 80% of the image is considered to ignore the floor and overhead obstacles that lie outside the drone’s path.
- Second, the bottom half of the image is weighed less heavily as it is mostly populated by the floor, which has higher depth values.
- Third, the 80th percentile of the pixels is taken to filter out outliers.
- Finally, the mean of the remaining pixels is taken to get a single danger value for each column.

See Figure 3 for an example of the input image, the associated depth map, the cropping regions, and the calculated danger values.

Note that the drone receives images in UYVY format, which is a packed format where each pixel is represented by two bytes. The first byte contains the Y value (luminance) and the second byte contains the UV values (chrominance). The images are converted to YUV format to ensure convolutional layers can be used, which is a conversion that can be performed at sub-ms speed on the drone. The images are also rotated by 90 degrees (clockwise) to align with the drone’s

actual camera orientation. The final dataset consisted of approximately 10,000 images.

4.3 Model Architecture, Training, and Evaluation

A three-layer convolutional network with three stages of convolution, ReLU activation, and pooling was implemented. After the convolutional blocks, the extracted features are flattened and passed through two fully connected layers to output the three danger values. The network is configurable to use either ‘standard’ or ‘efficient’ architecture. Standard architecture uses standard convolutional layers and two fully connected layers; efficient architecture uses depthwise separable convolutions [27] and an additional squeeze layer (halving the layer width) between the fully connected layers to potentially improve inference speed. Common hyperparameters such as learning rate (0.0002), dropout rate (0.1), gradient clipping (0.5), and weight decay (0.001) were set based on preliminary experiments and kept consistent across model variants to focus our comparison on architectural differences rather than training optimization. The model is trained using 3-fold cross-validation and includes data augmentation using small rotations and minor brightness and contrast adjustments, after downscaling the input by a factor of either 2 or 4. The training employs the Adam optimizer [28], OneCycleLR scheduler [29], Huber Loss ($\delta = 0.1$) to prevent fitting to the mean [30], and early stopping to prevent overfitting. Suitable configurations were found by testing untrained models on the drone and selecting configurations that showed inference speeds under 100 ms (though the trained models showed different inference speeds). As described in Subsection 3.4, to perform this test, the PyTorch models were converted to ONNX format and then to C using ONNX2C to use the model in the inference pipeline discussed in Subsection 4.4. Channel sizes of 4-32 and fully connected layer widths of 32-128 (in steps of powers of 2) were selected. Despite training small models for just three folds, training each configuration took approximately 2 hours due to the large dataset and limited GPU power. Hence, we limited training to a selection of 20 promising configurations and neglected an ablation study on the augmentations. After training, models that performed well on all folds were selected for a final speed benchmark on the drone. The four best configurations are summarized in Table 1.

Input	FC Size	Inf. (ms)	Huber Loss
60	128	17.98	0.00822
60	64	13.33	0.00931
120	128	78.69	0.00821
120	64	65.54	0.00888

Table 1: Performance comparison of top models. All models shown use depthwise convolutions, 8-16-32 channels in the three convolution blocks respectively, and include the optional squeeze layer between the final fully connected layers

The results in Table 1 show that models with an input size of 60 pixels achieve similar accuracy to their 120-pixel counterparts while requiring significantly less inference time (4-5× faster). The fully connected layer width appears to have similar effects, but less drastically, with the 128-neuron configuration showing slightly better performance. These findings suggest that aggressive downsampling to 60×60 pixels combined with a larger fully connected layer of 128 neurons provides a valid trade-off between accuracy and computational efficiency. Do note that a 20 ms inference time does not imply a 50 Hz operation, as the model must be run parallel to other modules on the drone. Another interesting finding is that while depthwise separable convolutions and squeeze layers are commonly associated with reduced parameter count - and thus reduced inference times, our experiments showed mostly reduced inference times and less consistently reduced parameter counts, suggesting the improved inference times are due to the reduced computational complexity of depthwise separable convolutions rather than the reduced parameter count. This indicates that the parameter count may not be a valid metric for predicting inference times in our use case. For a visual overview of the final model architecture, see Figure 4.

4.4 Inference pipeline

To perform inference on the drone, a paparazzi computer vision thread is registered using `cv_add_to_device()` to request image collection at 10 FPS without blocking essential control threads. Images are collected into a FIFO queue, which drops old frames when the queue fills up. Frames are then collected from the queue and prepared for inference in a separate processing thread, which calls the necessary preprocessing: conversion from the UYVY format to YUV, cropping to the center 240×240 pixels, and applying 4x downscaling in an efficient all-in-one function, taking 1-2 ms per frame. The preprocessed image is then fed into the model, and the output is then published in an ABI message. For a more detailed overview of the code, the reader is referred to our github repository²

4.5 Drone movement strategy

The steering module of the drone is designed as a finite state machine that transitions between four states: `SEARCH_FOR_SAFE_HEADING`, `SAFE`, `OUT_OF_BOUNDS`, and `REENTER_ARENA`. These transitions are triggered based on the inferred obstacle proximity. This state system is based on the `orange_avoider_guided.c` file's state system but is heavily modified to fit our movement strategy. When operating in the `SAFE` state, the drone moves forward while avoiding obstacles detected by the onboard perception model. The model outputs a 3-column danger array corresponding to the left, center, and right of the drone's field of view. Based on these values, a heading direction is computed

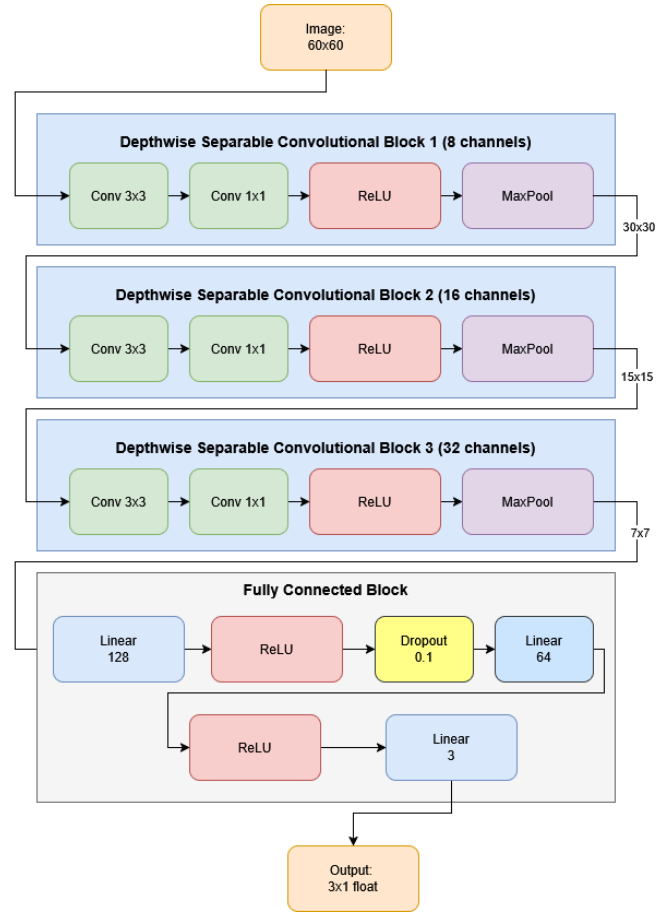


Figure 4: Final model architecture.

²<https://github.com/TwanvanDam/paparazzi>

to steer the drone away from high-risk areas. The drone slows down proportionally to the lowest danger score to promote cautious navigation near obstacles. In the presence of high danger, the drone stops moving and resorts to rotating to correctly handle the danger of imminent collision.

To ensure smooth direction changes and reduce oscillatory behavior, a linearly weighed moving average is applied to the heading direction using a heading buffer filter without adding significant latency. When the drone is considered to be outside the Cyber Zoo boundary, the state transitions to `OUT_OF_BOUNDS`. In this state, the drone halts forward movement and rotates in place until it detects it has re-entered the arena. It then enters the `REENTER_ARENA` state and turns around using OptiTrack data to correctly change the heading direction to fly inside the obstacle zone again.

This state-based reactive navigation ensures that the drone remains inside the arena and avoids collisions robustly and adaptively.

4.6 Limitations

While hyperdistillation enabled real-time performance, it simplified depth estimation into coarse danger zones. This occasionally led to collisions, as seen in the Cyber Zoo tests. Future work could refine the danger mapping by tuning the danger value calculation, expanding the number of columns, or developing a more sophisticated temporal filtering, e.g. the use of SLAM algorithms to aggregate depth maps over time. Additionally, adding rows in the danger value calculation could help to avoid collisions with overhead obstacles. The current approach also struggles with motion blur, which can be improved by using a more advanced teacher model such as Consistent Depth or Depth Anything V2.

5 TESTING CHALLENGES AND PRELIMINARY RESULTS

The system underwent three phases of testing: **simulation in Gazebo**, **static hardware benchmarks**, and **limited real-world flights in the Cyber Zoo**.

Simulation in Gazebo The provided Gazebo simulation environment provided a controlled setting to develop the C code to detect obstacles and test the steering of the drone. However, simulated images lack motion blur, lighting variability, texture, noise, and other real (camera) effects. Put simply, simulated images look nothing like the real world, creating a *reverse sim-real gap*. In other words, the model that was trained on real-world data had difficulties generalizing to the simulated environment. This is in contrast to the more common *sim-real gap*, where a model trained in simulation fails to generalize to the real world. The Gazebo simulation was used to test the steering of the drone and the object avoidance algorithms, but not for training. Methods to overcome this were discussed in Section 2, but too complex to be implemented in the limited timeframe of the project.

Despite this, the final implementation showed reasonable results in the simulator, but fine-tuning the navigation parameters showed little transfer to the real world.

Static Hardware Benchmarks At-home testing was performed using socket power - not enabling flight, but allowing for testing the speed of the inference pipeline and calculation of steering commands on the drone hardware. Telemetry messages from the `sysmon` module (a default module in Paparazzi that monitors aspects of the control loop) were used to verify that our modules did not impede on critical control loop performance.

Cyber Zoo Testing Due to organisational constraints, the Cyber Zoo was only available for a single half-hour slot of flight time per week, with two extra sessions in the final week of the project. This, along with issues with the OptiTrack system when multiple drones were flying at the same time, resulted in flight-breaking bugs being discovered later than necessary and severely limited tuning opportunities. Hence, a proper validation of the system during flight was not possible.

6 RESULTS

During the competition, the drone flew 350 meters, which was the longest distance among the twelve groups participating in the course; however, it also experienced several collisions. The drone was awarded **second place** overall in the competition. As saving or streaming images during flight is a time-consuming process, we disabled this during the final competition, so we could not properly evaluate the major failing point of the drone during the competition. However, in this report, we have already identified potential failing points and proposed solutions, summarized as follows:

- The distillation and training method could be calibrated to use a different weighing or cropping strategy, a different number of rows and columns, or a more accurate, slower model.
- Additional teacher models could be explored to enhance the training data.
- More sophisticated methods of incorporating the drone’s state and history of measurements could be explored, such as using SLAM methods or simple Kalman or particle filtering.
- Solutions to bridge the sim-real and real-sim gap could be explored further, such as using a visually realistic 3D mapping of the Cyber Zoo.

7 CONCLUSION

While the developed module has shown promising results, there is still room for improvement. In this report, we

have provided a detailed overview of the methods used to train the drone to detect obstacles and navigate through the Cyber Zoo. We have also discussed the challenges faced during the competition and proposed potential solutions to improve the performance of the drone in future iterations. Overall, we believe that with further refinement and exploration of new methods, the drone can achieve even better results in obstacle detection and navigation tasks. Additionally, we find that the course has been both a valuable and enjoyable learning experience, providing real-life experience with the challenges and opportunities in the field of autonomous drone navigation. We want to thank the teachers and teaching assistants for their time and energy and look forward to applying these lessons in future projects and competitions.

REFERENCES

- [1] Pascal Brisset, Antoine Drouin, Michel Gorraz, Pierre-Selim Huard, and Jeremy Tyler. The paparazzi solution. In *MAV 2006, 2nd US-European competition and workshop on micro air vehicles*, 2006.
- [2] René Ranftl, Katrin Lasinger, David Hafner, Konrad Schindler, and Vladlen Koltun. Towards robust monocular depth estimation: Mixing datasets for zero-shot cross-dataset transfer. *IEEE transactions on pattern analysis and machine intelligence*, 44(3):1623–1637, 2020.
- [3] Weihao Yuan, Xiaodong Gu, Zuozhuo Dai, Siyu Zhu, and Ping Tan. New crfs: Neural window fully-connected crfs for monocular depth estimation, 2022.
- [4] Lihe Yang, Bingyi Kang, Zilong Huang, Xiaogang Xu, Jiashi Feng, and Hengshuang Zhao. Depth anything: Unleashing the power of large-scale unlabeled data. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 10371–10381, 2024.
- [5] Lihe Yang, Bingyi Kang, Zilong Huang, Zhen Zhao, Xiaogang Xu, Jiashi Feng, and Hengshuang Zhao. Depth anything v2. *Advances in Neural Information Processing Systems*, 37:21875–21911, 2024.
- [6] Xuan Luo, Jia-Bin Huang, Richard Szeliski, Kevin Matzen, and Johannes Kopf. Consistent video depth estimation. 39(4), 2020.
- [7] Johannes Kopf, Xuejian Rong, and Jia-Bin Huang. Robust consistent video depth estimation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 1611–1621, 2021.
- [8] Daniele Palossi, Nicky Zimmerman, Alessio Burrello, Francesco Conti, Hanna Müller, Luca Maria Gambardella, Luca Benini, Alessandro Giusti, and Jérôme Guzzi. Fully onboard ai-powered human-drone pose estimation on ultra-low power autonomous flying nano-uavs. 3 2021.
- [9] Hong Cai, Fei Yin, Tushar Singhal, Parham Noorzad, Yinhao Zhu, Khoi Nguyen, Janarbek Matai, Bharath Ramaswamy, Frank Mayer, Chirag Patel, Abhijit Khobare, Fatih Porikli, Qualcomm AI Research, Douwe Kiela, Marco Ciccone, and Barbara Caputo. Real-time and accurate self-supervised monocular depth estimation on mobile device. Technical report, 2022.
- [10] Xiaohan Tu, Cheng Xu, Siping Liu, Renfa Li, Guoqi Xie, Jing Huang, and Laurence Tianruo Yang. Efficient monocular depth estimation for edge devices in internet of things. *IEEE Transactions on Industrial Informatics*, 17:2821–2832, 4 2021.
- [11] Antonio Cipolletta, Valentino Peluso, Andrea Calimera, Matteo Poggi, Fabio Tosi, Filippo Aleotti, and Stefano Mattoccia. Energy-quality scalable monocular depth estimation on low-power cpus. *IEEE Internet of Things Journal*, 9:25–36, 1 2022.
- [12] Jun Liu, Qing Li, Rui Cao, Wenming Tang, and Guoping Qiu. Mininet: An extremely lightweight convolutional neural network for real-time unsupervised monocular depth estimation. 6 2020.
- [13] Lorenzo Papa, Gabriele Proietti Mattia, Paolo Russo, Irene Amerini, and Roberto Beraldi. Lightweight and energy-aware monocular depth estimation models for iot embedded devices: Challenges and performances in terrestrial and underwater scenarios †. *Sensors*, 23, 2 2023.
- [14] Valentino Peluso, Antonio Cipolletta, Andrea Calimera, Matteo Poggi, Fabio Tosi, Filippo Aleotti, and Stefano Mattoccia. Monocular depth perception on microcontrollers for edge applications. *IEEE Transactions on Circuits and Systems for Video Technology*, 32:1524–1536, 3 2022.
- [15] Danish Ansari, Damian Bhawan, Patrick Kostelac, David Schep, Manu Singh, and Gijs Zijdeveld. Cnn-based autonomous navigation for micro air vehicles. Technical report, 2024.
- [16] Junjie Hu, Chenyou Fan, Hualie Jiang, Xiyue Guo, Yuan Gao, Xiangyong Lu, and Tin Lun Lam. Boosting light-weight depth estimation via knowledge distillation. 5 2021.
- [17] Paula Fraga-Lamas, Lucía Ramos, Víctor Mondéjar-Guerra, and Tiago M Fernández-Caramés. A review on iot deep learning uav systems for autonomous obstacle detection and collision avoidance. *Remote Sensing*, 11(18):2144, 2019.

- [18] Fereshteh Sadeghi and Sergey Levine. Cad2rl: Real single-image flight without a single real image. *arXiv preprint arXiv:1611.04201*, 2016.
- [19] Dong Ki Kim and Tsuhan Chen. Deep neural network for real-time autonomous indoor navigation. *arXiv preprint arXiv:1511.04668*, 2015.
- [20] Zhizhong Kang, Juntao Yang, Zhou Yang, and Sai Cheng. A review of techniques for 3d reconstruction of indoor environments. *ISPRS International Journal of Geo-Information*, 9(5):330, 2020.
- [21] Xinjian Fang, Yingdan Zhang, Hao Tan, Chao Liu, and Xu Yang. Performance evaluation and optimization of 3d gaussian splatting in indoor scene generation and rendering. *ISPRS International Journal of Geo-Information*, 14(1):21, 2025.
- [22] Alex Fisher, Ricardo Cannizzaro, Madeleine Cochrane, Chatura Nagahawatte, and Jennifer L Palmer. Colmap: A memory-efficient occupancy grid mapping framework. *Robotics and Autonomous Systems*, 142:103755, 2021.
- [23] ai-edge bot, Luke Boyer, Terry Heo, Quentin Khan, Jun Jiang, Ezekiel C, YunanAZ, gaikwadrahul8, Francisco Unda, Alen Huang, chunhsue, Steven Toribio, Fergus Henderson, alankelly, Chunlei Niu, chuntl, zwei, Weiyi Wang, Wai Hon Law, Vitalii Dziuba, Tilak, Reilly Grant, Majid Dadashi, pak laura, C. Antonio Sánchez, Oleh Prypin, Peter Hawkins, Reed, Tommy Chiang, and Vamsi Krishna Manchala. google-ai-edge/litert, 3 2025.
- [24] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, et al. {TVM}: An automated {End-to-End} optimizing compiler for deep learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, pages 578–594, 2018.
- [25] kraiskil, tlane rbx, can-leh emmtrix, Robin van Emden, YoungsunUConn, mryndzzonek, AUTOMATIC1111, Björn Ganster, Isak Nyberg, Kern Handa, Peter Bohm, Stuart Watt, and YJack0000. *kraiskil/onnx2c*. 1 2025.
- [26] Alessandro Cerioli, Riccardo Miccini, Clément Laroche, Tobias Piechowiak, Luca Pezzarossa, Jens Sparsø, and Martin Schoeberl. Neuralcasting: a front-end compilation infrastructure for neural networks. In *2024 11th International Conference on Internet of Things: Systems, Management and Security (IOTSMS)*, pages 161–168. IEEE, 2024.
- [27] François Chollet. Xception: Deep learning with depth-wise separable convolutions. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 1251–1258, 2017.
- [28] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- [29] Leslie N Smith and Nicholay Topin. Super-convergence: Very fast training of neural networks using large learning rates. In *Artificial intelligence and machine learning for multi-domain operations applications*, volume 11006, pages 369–386. SPIE, 2019.
- [30] Peter J Huber. Robust estimation of a location parameter. In *Breakthroughs in statistics: Methodology and distribution*, pages 492–518. Springer, 1992.

APPENDIX A: FLOWCHART OF AVOIDER.C STRUCTURE

